



Phase 2 - São Paulo Deployment Code (Sanitized)



VALUES YOU'LL NEED FROM PHASE 1 (run these after Tokyo apply completes):

```
terraform output tokyo_tgw_id  
terraform output tokyo_rds_endpoint
```

Add these to your São Paulo terraform.tfvars:

- tokyo_tgw_id = "tgw-XXXXXXXXXXXXXXXXXXXX" ← from output above
- tokyo_rds_endpoint = "helga-rdslab3.XXXXXXXXXX.ap-northeast-1.rds.amazonaws.com:3306" ← from output above

This phase deploys São Paulo infrastructure. It's **Lab 2 pattern without RDS** plus the Transit Gateway spoke.



CRITICAL - APPI COMPLIANCE: São Paulo has **NO RDS**. All database access goes to Tokyo via TGW.

CIDR Reference

Region	Resource	CIDR
Tokyo	VPC	10.0.0.0/16
Tokyo	Public Subnets	10.0.1.0/24, 10.0.2.0/24
Tokyo	Private Subnets	10.0.11.0/24, 10.0.12.0/24
São Paulo	VPC	10.1.0.0/16

Region	Resource	CIDR
São Paulo	Public Subnets	10.1.1.0/24, 10.1.2.0/24
São Paulo	Private Subnets	10.1.11.0/24, 10.1.12.0/24

São Paulo Directory Setup

Create a separate directory for São Paulo (separate Terraform state):

```
mkdir -p lab3/saopaulo
cd lab3/saopaulo
```

File 1: 01-version.tf (NEW FILE)

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 5.0"
    }
  }
}
```

File 2: 02-providers.tf (NEW FILE)

```
# São Paulo Liberdade - Default provider
provider "aws" {
  region = "sa-east-1"
}
```

File 3: 03-variables.tf (NEW FILE)

```
#####  
# SÃO PAULO VARIABLES  
#####  
  
variable "aws_region" {  
    description = "AWS Region for São Paulo"  
    type        = string  
    default     = "sa-east-1"  
}  
  
variable "project_name" {  
    description = "Project name prefix"  
    type        = string  
    default     = "liberdade"  
}  
  
variable "vpc_cidr" {  
    description = "São Paulo VPC CIDR"  
    type        = string  
    default     = "10.1.0.0/16"  
}  
  
variable "public_subnet_cidrs" {  
    description = "São Paulo public subnet CIDRs (for NAT Gateway)"  
    type        = list(string)  
    default     = ["10.1.1.0/24", "10.1.2.0/24"]  
}  
  
variable "private_subnet_cidrs" {  
    description = "São Paulo private subnet CIDRs (for TGW attachment)"  
    type        = list(string)
```

```

    default      = ["10.1.11.0/24", "10.1.12.0/24"]
}

variable "azs" {
    description = "São Paulo AZs"
    type        = list(string)
    default     = ["sa-east-1a", "sa-east-1b"]
}

# Cross-region references (from Tokyo Phase 1 outputs)
variable "tokyo_vpc_cidr" {
    description = "Tokyo VPC CIDR for routing"
    type        = string
    default     = "10.0.0.0/16"
}

variable "tokyo_tgw_id" {
    description = "Tokyo TGW ID (from Phase 1 output)"
    type        = string
    # Set in terraform.tfvars after Phase 1
}

variable "tokyo_rds_endpoint" {
    description = "Tokyo RDS endpoint (from Phase 1 output)"
    type        = string
    sensitive   = true
    default     = ""
    # Set in terraform.tfvars
}

variable "enable_tgw" {
    description = "Enable Transit Gateway for Lab 3"
    type        = bool
    default     = true
}

```

File 4: 04-main.tf (NEW FILE - VPC + Network, NO RDS)

Part 1: VPC + IGW + Subnets

```
locals {
  name_prefix = var.project_name
}

resource "aws_vpc" "liberdade_vpclab3" {
  cidr_block      = var.vpc_cidr
  enable_dns_support = true
  enable_dns_hostnames = true
  tags = {
    Name      = "liberdade-vpclab3"
    Compliance = "APPI-compute-only"
  }
}

resource "aws_internet_gateway" "liberdade_igwlab3" {
  vpc_id = aws_vpc.liberdade_vpclab3.id
  tags = { Name = "liberdade-igwlab3" }
}

resource "aws_subnet" "liberdade_public_subnets" {
  count                = length(var.public_subnet_cidrs)
  vpc_id              = aws_vpc.liberdade_vpclab3.id
  cidr_block          = var.public_subnet_cidrs[count.index]
  availability_zone    = var.azs[count.index]
  map_public_ip_on_launch = true
  tags = { Name = "liberdade-public-subnet0${count.index + 1}" }
}
```

```

resource "aws_subnet" "liberdade_private_subnets" {
  count          = length(var.private_subnet_cidrs)
  vpc_id         = aws_vpc.liberdade_vpclab3.id
  cidr_block     = var.private_subnet_cidrs[count.index]
  availability_zone = var.azs[count.index]
  tags = { Name = "liberdade-private-subnet0${count.index +
1}" }
}

```

Part 2: NAT Gateway

```

resource "aws_eip" "liberdade_nat_eiplab3" {
  domain = "vpc"
  tags = { Name = "liberdade-nat-eiplab3" }
}

resource "aws_nat_gateway" "liberdade_natlab3" {
  allocation_id = aws_eip.liberdade_nat_eiplab3.id
  subnet_id     = aws_subnet.liberdade_public_subnets[0].id
  tags = { Name = "liberdade-natlab3" }
  depends_on = [aws_internet_gateway.liberdade_igwlab3]
}

```

Part 3: Route Tables

```

resource "aws_route_table" "liberdade_public_rtlab3" {
  vpc_id = aws_vpc.liberdade_vpclab3.id
  tags = { Name = "liberdade-public-rtlalab3" }
}

resource "aws_route" "liberdade_public_default_route" {
  route_table_id = aws_route_table.liberdade_public_rtlab3.id
  destination_cidr_block = "0.0.0.0/0"
}

```

```

    gateway_id          = aws_internet_gateway.liberdade_igw
lab3.id
}

resource "aws_route_table_association" "liberdade_public_rta"
{
    count                = length(aws_subnet.liberdade_public_subnet
s)
    subnet_id           = aws_subnet.liberdade_public_subnets[count.
index].id
    route_table_id      = aws_route_table.liberdade_public_rtlab3.id
}

resource "aws_route_table" "liberdade_private_rtlab3" {
    vpc_id = aws_vpc.liberdade_vpclab3.id
    tags = { Name = "liberdade-private-rtl3" }
}

resource "aws_route" "liberdade_private_default_route" {
    route_table_id      = aws_route_table.liberdade_private_
rtl3.id
    destination_cidr_block = "0.0.0.0/0"
    nat_gateway_id      = aws_nat_gateway.liberdade_natlab3.
id
}

resource "aws_route_table_association" "liberdade_private_rt
a" {
    count                = length(aws_subnet.liberdade_private_subnet
s)
    subnet_id           = aws_subnet.liberdade_private_subnets[coun
t.index].id
    route_table_id      = aws_route_table.liberdade_private_rtl3.i
d
}

```

File 5: 04-P-main-saopaulo-tgw.tf (NEW FILE)

TGW Resource

```
resource "aws_ec2_transit_gateway" "liberdade_tgwlab3" {
  count          = var.enable_tgw ? 1 : 0
  description    = "liberdade-tgwlab3 (São Paulo spoke)"

  amazon_side_asn            = 64513
  auto_accept_shared_attachments = "disable"
  default_route_table_association = "enable"
  default_route_table_propagation = "enable"
  dns_support                 = "enable"
  vpn_ecmp_support            = "enable"

  tags = {
    Name     = "liberdade-tgwlab3"
    Role     = "Spoke"
    Project  = var.project_name
  }
}
```

VPC Attachment (Private Subnets Only)

```
resource "aws_ec2_transit_gateway_vpc_attachment" "liberdade_
attach_sp_vpclab3" {
  count          = var.enable_tgw ? 1 : 0
  transit_gateway_id = aws_ec2_transit_gateway.liberdade_tgwlab3[0].id
  vpc_id          = aws_vpc.liberdade_vpclab3.id

  subnet_ids = [
    aws_subnet.liberdade_private_subnets[0].id,
    aws_subnet.liberdade_private_subnets[1].id
  ]
}
```



```

]

dns_support                                = "enable"
transit_gateway_default_route_table_association = true
transit_gateway_default_route_table_propagation = true

tags = {
  Name      = "liberdade-attach-sp-vpclab3"
  Project = var.project_name
}
}

```

File 6: 05-outputs.tf (NEW FILE)

```

output "saopaulo_vpc_id" {
  description = "São Paulo VPC ID"
  value       = aws_vpc.liberdade_vpclab3.id
}

output "saopaulo_vpc_cidr" {
  description = "São Paulo VPC CIDR"
  value       = aws_vpc.liberdade_vpclab3.cidr_block
}

output "saopaulo_public_subnet_ids" {
  description = "São Paulo public subnet IDs"
  value       = aws_subnet.liberdade_public_subnets[*].id
}

output "saopaulo_private_subnet_ids" {
  description = "São Paulo private subnet IDs"
  value       = aws_subnet.liberdade_private_subnets[*].id
}

```

```

output "saopaulo_tgw_id" {
  description = "São Paulo Transit Gateway ID (for Tokyo peer
ing)"
  value      = var.enable_tgw ? aws_ec2_transit_gateway.lib
erdade_tgwlab3[0].id : null
}

output "saopaulo_tgw_arn" {
  description = "São Paulo Transit Gateway ARN"
  value      = var.enable_tgw ? aws_ec2_transit_gateway.lib
erdade_tgwlab3[0].arn : null
}

```

File 7: **terraform.tfvars** (CREATE AFTER PHASE 1 COMPLETES)

```

tokyo_tgw_id      = "tgw-XXXXXXXXXXXXXXXXXXXX"
tokyo_rds_endpoint = "helga-rdslab3.XXXXXXXXXX.ap-northeast-1.
rds.amazonaws.com:3306"

```

Deployment Commands (Phase 2)

```

cd lab3/saopaulo
terraform init
terraform validate
terraform plan -out phase2
terraform apply phase2
terraform output saopaulo_tgw_id

```

Verification Checklist (Phase 2)

- ☐ São Paulo VPC created (liberdade-vpclab3) with CIDR 10.1.0.0/16

- ☐ 2 public subnets (10.1.1.0/24, 10.1.2.0/24)
- ☐ 2 private subnets (10.1.11.0/24, 10.1.12.0/24)
- ☐ NAT Gateway operational in public subnet
- ☐ São Paulo TGW created (liberdade-tgwlab3)
- ☐ TGW attached to private subnets
- ☐ **NO RDS exists in São Paulo** (CRITICAL)

```
aws rds describe-db-instances --region sa-east-1
aws ec2 describe-vpcs --region sa-east-1 --filters "Name=tag:
Name,Values=*liberdade*"
aws ec2 describe-transit-gateways --region sa-east-1
aws ec2 describe-transit-gateway-vpc-attachments --region sa-
east-1
```



Phase 2 Complete: Capture saopaulo_tgw_id output, then proceed to Phase 3 to create TGW peering from Tokyo.